ACADEMY OF TECHNOLOGY
... translate your vision into reality

Operating System (PCC-CS502/ PCC-CS592)

Assignment-I

Name: Monojit Goswami

Roll: 16900124112

Semester: 5th

Year: 2026

Academy of Technology

G. T. Road (Adisaptagram), Aedconagar
Hooghly-712121 West Bengal, India

Project 1: Snake Game Using Shell Script

Problem statement: Design and implement a Snake Game using a shell script. The game provides an interactive terminal-based interface in which the snake moves in different directions, grows after eating food, and ends when it collides with itself or a boundary. The script handles input, game logic, score tracking, and terminal display updates.

Objective

To implement a playable terminal arcade game using Bash, with keyboard controls, a visible game arena, score tracking, difficulty settings, color themes, and game-state screens.

Features and controls

- Keyboard movement using arrow keys, WASD, and Vim-style HJKL controls.
- Score and high-score tracking, with a persistent high-score file in the user home directory.
- Multiple difficulty levels and color themes, plus selectable wall behavior.
- Pause, instructions, restart, main-menu, and game-over screens.

How to execute

```
chmod +x snake.sh
./snake.sh
```

Program output

The following screenshots were captured from the program running in an interactive terminal instance.

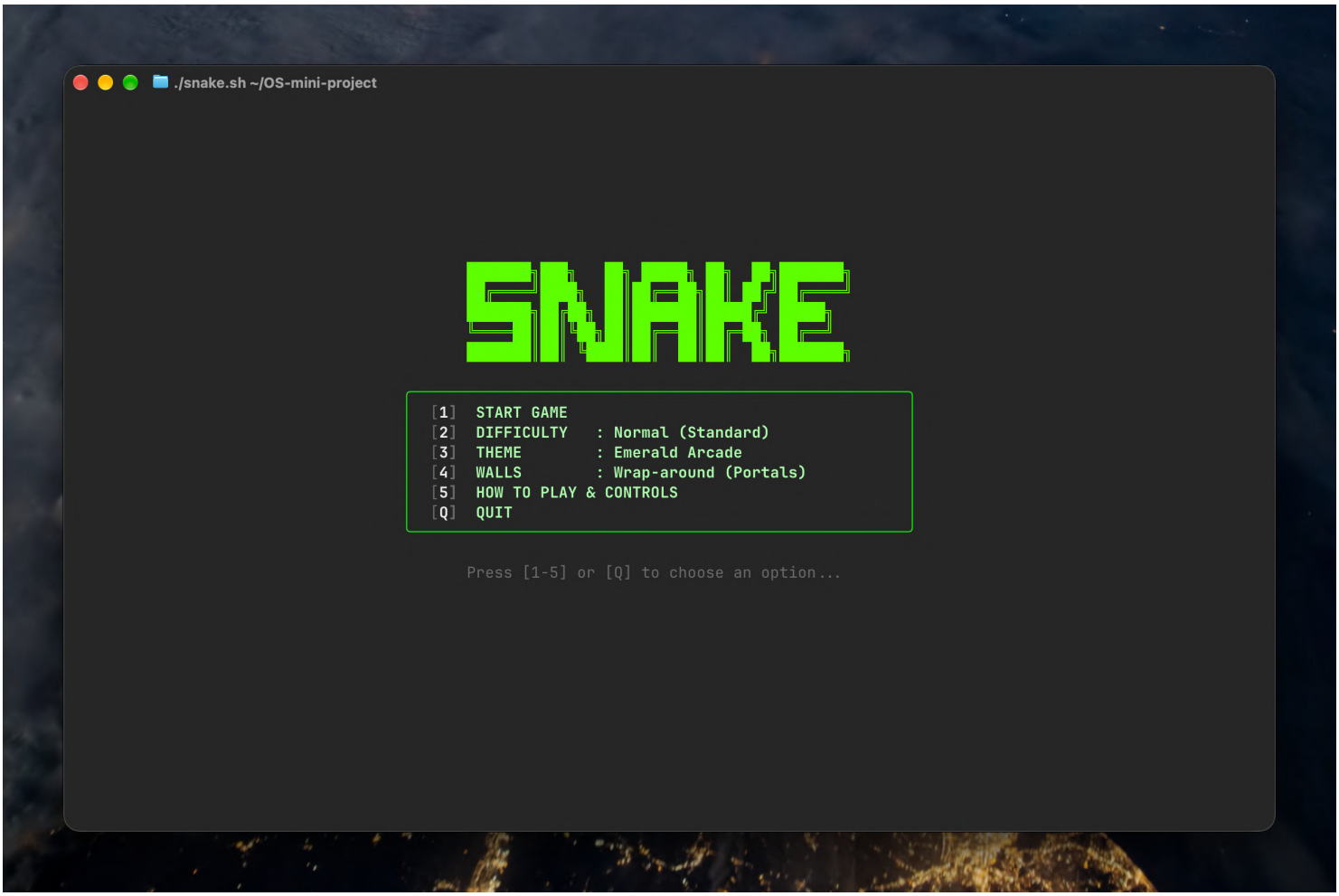


Figure 1. Terminal output of Project 1, Snake Game main menu.

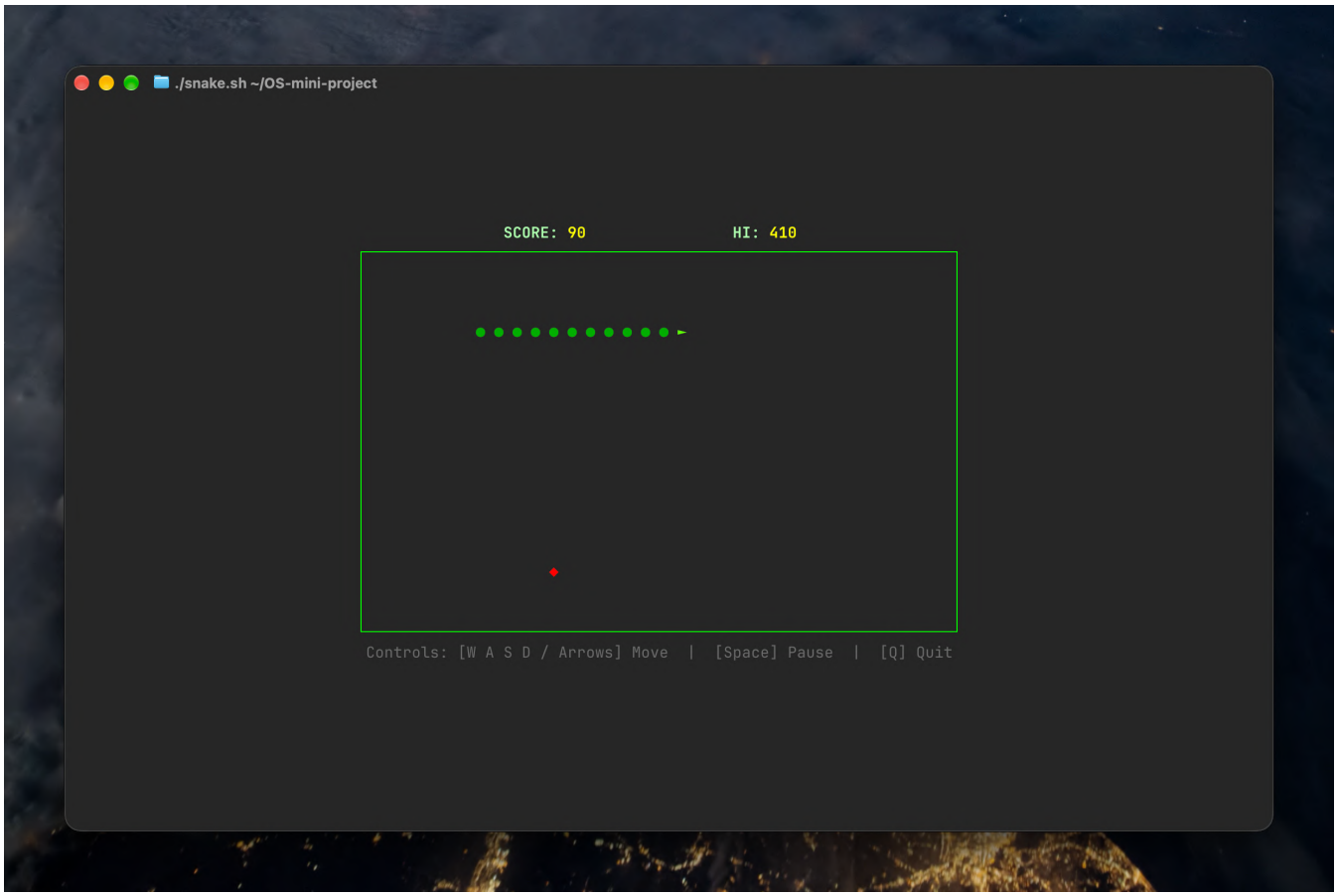


Figure 2. Terminal output of Project 1, Snake Game Live Gameplay



Figure 3. Terminal output of Project 1, Game Over Screen.

Source Code: snake.sh

```
#!/usr/bin/env bash

set -u

# Single-byte ASCII Escape code for ANSI terminal formatting (compatible with all shells and printf %s)
ESC=$(printf '\033')

# High score file location (prefers user home directory, fallbacks to local)
HIGHSCORE_FILE="${HOME}/.snake_highscore"
if [ ! -w "$(dirname "$HIGHSCORE_FILE")" ]; then
    HIGHSCORE_FILE="./snake_highscore"
fi

# Terminal cleanup handler
ORIG_STTY=""
cleanup() {
    printf "%s[?25h" "$ESC"          # Show cursor
    printf "%s[0m" "$ESC"            # Reset color styles
    printf "%s[2J%s[H" "$ESC" "$ESC" # Clear screen and reset cursor home
    if [ -n "$ORIG_STTY" ]; then
        stty "$ORIG_STTY" 2>/dev/null || true
    fi
    exit 0
}

trap cleanup INT TERM EXIT

# Save original terminal settings and enable raw non-blocking mode
ORIG_STTY=$(stty -g 2>/dev/null || true)
stty -echo -icanon min 0 time 0 2>/dev/null || true
printf "%s[?25l" "$ESC" # Hide cursor

# ------------------------------------------------------------------------------
# Game Configuration & Global State
# ------------------------------------------------------------------------------
BOARD_W=32            # Grid cells wide (each cell = 2 terminal columns → 64 cols)
BOARD_H=18            # Grid cells high (18 rows high → 16:10 visual aspect ratio)
CELL_WIDTH=2          # 2 characters per cell for square cell geometry

DIFFICULTY=1          # 0 = Casual (0.12s), 1 = Normal (0.08s), 2 = Hard (0.05s), 3 = Insane (0.035s)
WALL_MODE=0           # 0 = Wrap-around (portal walls), 1 = Solid walls (die on crash)
THEME=0               # 0 = Emerald Arcade, 1 = Cyberpunk Neon, 2 = Synthwave, 3 = Ice Blue, 4 = Monochrome

MIN_COLS=68
MIN_ROWS=24

update_term_geometry() {
    local size
    size=$(stty size 2>/dev/null || echo "24 80")
    read -r TERM_ROWS TERM_COLS <<< "$size"
    [ -z "$TERM_ROWS" ] || [ "$TERM_ROWS" -le 0 ] && TERM_ROWS=24
    [ -z "$TERM_COLS" ] || [ "$TERM_COLS" -le 0 ] && TERM_COLS=80

    if [ "$TERM_COLS" -lt "$MIN_COLS" ] || [ "$TERM_ROWS" -lt "$MIN_ROWS" ]; then
        printf "\033[2J\033[H"
        printf "\033[1;38;5;196m⚠️  TERMINAL WINDOW TOO SMALL\033[0m\n\n"
        printf "Current size : %d cols x %d rows\n" "$TERM_COLS" "$TERM_ROWS"
        printf "Minimum required: %d cols x %d rows\n\n" "$MIN_COLS" "$MIN_ROWS"
        printf "Please resize or zoom out your terminal, then press any key...\n"
        dd bs=10 count=1 2>/dev/null || true
    fi

    # Auto-center game arena on terminal
    local total_w=$(( BOARD_W * CELL_WIDTH + 2 ))
    local total_h=$(( BOARD_H + 4 )) # 1 HUD + 2 arena borders + 1 controls

    OFFSET_X=$(( (TERM_COLS - total_w) / 2 ))
    OFFSET_Y=$(( (TERM_ROWS - total_h) / 2 ))
    [ "$OFFSET_X" -lt 1 ] && OFFSET_X=1
    [ "$OFFSET_Y" -lt 1 ] && OFFSET_Y=1
}

# ------------------------------------------------------------------------------
# Color Palettes
# ------------------------------------------------------------------------------
init_theme() {
    case "$THEME" in
        0) # Emerald Arcade (Default)
            COLOR_BORDER="${ESC}[38;5;46m"
            COLOR_HUD_TXT="${ESC}[1;38;5;157m"
            COLOR_HUD_VAL="${ESC}[1;38;5;226m"
            COLOR_HEAD="${ESC}[1;38;5;82m"
            COLOR_BODY="${ESC}[38;5;34m"
            COLOR_FOOD="${ESC}[1;38;5;196m"
            COLOR_MODAL="${ESC}[1;38;5;46m"
            THEME_NAME="Emerald Arcade"
            ;;
        1) # Cyberpunk Neon
            COLOR_BORDER="${ESC}[38;5;51m"
            COLOR_HUD_TXT="${ESC}[1;38;5;213m"
            COLOR_HUD_VAL="${ESC}[1;38;5;226m"
            COLOR_HEAD="${ESC}[1;38;5;201m"
            COLOR_BODY="${ESC}[38;5;45m"
            COLOR_FOOD="${ESC}[1;38;5;196m"
```

```
            COLOR_MODAL="${ESC}[1;38;5;51m"
            THEME_NAME="Cyberpunk Neon"
            ;;
        2) # Synthwave Sunset
            COLOR_BORDER="${ESC}[38;5;198m"
            COLOR_HUD_TXT="${ESC}[1;38;5;222m"
            COLOR_HUD_VAL="${ESC}[1;38;5;208m"
            COLOR_HEAD="${ESC}[1;38;5;214m"
            COLOR_BODY="${ESC}[38;5;162m"
            COLOR_FOOD="${ESC}[1;38;5;51m"
            COLOR_MODAL="${ESC}[1;38;5;198m"
            THEME_NAME="Synthwave Sunset"
            ;;
        3) # Ice Blue
            COLOR_BORDER="${ESC}[38;5;39m"
            COLOR_HUD_TXT="${ESC}[1;38;5;153m"
            COLOR_HUD_VAL="${ESC}[1;38;5;87m"
            COLOR_HEAD="${ESC}[1;38;5;159m"
            COLOR_BODY="${ESC}[38;5;32m"
            COLOR_FOOD="${ESC}[1;38;5;208m"
            COLOR_MODAL="${ESC}[1;38;5;39m"
            THEME_NAME="Ice Blue"
            ;;
        4) # Retro Monochrome
            COLOR_BORDER="${ESC}[38;5;250m"
            COLOR_HUD_TXT="${ESC}[1;38;5;255m"
            COLOR_HUD_VAL="${ESC}[1;38;5;255m"
            COLOR_HEAD="${ESC}[1;38;5;255m"
            COLOR_BODY="${ESC}[38;5;244m"
            COLOR_FOOD="${ESC}[1;38;5;255m"
            COLOR_MODAL="${ESC}[1;38;5;255m"
            THEME_NAME="Monochrome CRT"
            ;;
    esac
    COLOR_RESET="${ESC}[0m"
    COLOR_DIM="${ESC}[38;5;242m"
    COLOR_GOLD="${ESC}[1;38;5;220m"
    COLOR_RED="${ESC}[1;38;5;196m"
    COLOR_WHITE="${ESC}[1;38;5;255m"
}

# ------------------------------------------------------------------------------
# High Score Management
# ------------------------------------------------------------------------------
get_high_score() {
    if [ -f "$HIGHSCORE_FILE" ]; then
        local hs
        hs=$(tr -cd '0-9' < "$HIGHSCORE_FILE" 2>/dev/null)
        if [ -n "$hs" ]; then
            echo "$hs"
            return
        fi
    fi
    echo "0"
}

save_high_score() {
    local score="$1"
    local current_hs
    current_hs=$(get_high_score)
    if [ "$score" -gt "$current_hs" ]; then
        echo "$score" > "$HIGHSCORE_FILE" 2>/dev/null || true
    fi
}

# ------------------------------------------------------------------------------
# How To Play Screen
# ------------------------------------------------------------------------------
show_instructions() {
    printf "\033[2J"
    update_term_geometry
    init_theme

    local box_w=62
    local inner_w=60
    local start_c=$(( (TERM_COLS - box_w) / 2 ))
    [ "$start_c" -lt 1 ] && start_c=1
    local row=$(( (TERM_ROWS - 17) / 2 ))
    [ "$row" -lt 1 ] && row=1

    local hline=""
    for ((i=0; i<inner_w; i++)); do hline="${hline}─"; done

    printf "%s[%d;%dH%s┌%s┐%s" "$ESC" "$row" "$start_c" "$COLOR_BORDER" "$hline" "$COLOR_RESET"; row=$((row+1))
    printf "%s[%d;%dH%s│%s                    HOW TO PLAY & CONTROLS                    %s│%s" "$ESC" "$row" "$start_c" "$COLOR_BORDER" "$COLOR_WHITE"
"$COLOR_BORDER" "$COLOR_RESET"; row=$((row+1))
    printf "%s[%d;%dH%s├%s┤%s" "$ESC" "$row" "$start_c" "$COLOR_BORDER" "$hline" "$COLOR_RESET"; row=$((row+1))
    printf "%s[%d;%dH%s│%s  MOVEMENT:                                                  %s│%s" "$ESC" "$row" "$start_c" "$COLOR_BORDER" "$COLOR_GOLD"
"$COLOR_BORDER" "$COLOR_RESET"; row=$((row+1))
    printf "%s[%d;%dH%s│%s    • Arrow Keys (Up, Down, Left, Right)                   %s│%s" "$ESC" "$row" "$start_c" "$COLOR_BORDER" "$COLOR_HUD_TXT"
"$COLOR_BORDER" "$COLOR_RESET"; row=$((row+1))
    printf "%s[%d;%dH%s│%s    • WASD Keys  (W=Up, A=Left, S=Down, D=Right)           %s│%s" "$ESC" "$row" "$start_c" "$COLOR_BORDER" "$COLOR_HUD_TXT"
"$COLOR_BORDER" "$COLOR_RESET"; row=$((row+1))
    printf "%s[%d;%dH%s│%s    • Vim Keys   (K=Up, H=Left, J=Down, L=Right)           %s│%s" "$ESC" "$row" "$start_c" "$COLOR_BORDER" "$COLOR_HUD_TXT"
"$COLOR_BORDER" "$COLOR_RESET"; row=$((row+1))
    printf "%s[%d;%dH%s│%s                                                          %s│%s" "$ESC" "$row" "$start_c" "$COLOR_BORDER" "$COLOR_RESET"
"$COLOR_BORDER" "$COLOR_RESET"; row=$((row+1))
    printf "%s[%d;%dH%s│%s  OBJECTIVES & ITEMS:                                     %s│%s" "$ESC" "$row" "$start_c" "$COLOR_BORDER" "$COLOR_GOLD"
```

```
"$COLOR_BORDER" "$COLOR_RESET"; row=$((row+1))
    printf "%s[%d;%dH%s│%s    • %s◆%s Fruit : +10 pts, grows snake by 1 unit             %s│%s" "$ESC" "$row" "$start_c" "$COLOR_BORDER"
"$COLOR_HUD_TXT" "$COLOR_FOOD" "$COLOR_HUD_TXT" "$COLOR_BORDER" "$COLOR_RESET"; row=$((row+1))
    printf "%s[%d;%dH%s│%s    • Avoid hitting your own body or boundary walls!        %s│%s" "$ESC" "$row" "$start_c" "$COLOR_BORDER" "$COLOR_HUD_TXT"
"$COLOR_BORDER" "$COLOR_RESET"; row=$((row+1))
    printf "%s[%d;%dH%s│%s                                                          %s│%s" "$ESC" "$row" "$start_c" "$COLOR_BORDER" "$COLOR_RESET"
"$COLOR_BORDER" "$COLOR_RESET"; row=$((row+1))
    printf "%s[%d;%dH%s│%s  ACTIONS:                                                %s│%s" "$ESC" "$row" "$start_c" "$COLOR_BORDER" "$COLOR_GOLD"
"$COLOR_BORDER" "$COLOR_RESET"; row=$((row+1))
    printf "%s[%d;%dH%s│%s    • [Space] Pause Game       |   [Q] Quit / Return        %s│%s" "$ESC" "$row" "$start_c" "$COLOR_BORDER" "$COLOR_HUD_TXT"
"$COLOR_BORDER" "$COLOR_RESET"; row=$((row+1))
    printf "%s[%d;%dH%s├%s┤%s" "$ESC" "$row" "$start_c" "$COLOR_BORDER" "$hline" "$COLOR_RESET"; row=$((row+1))
    printf "%s[%d;%dH%s│%s                Press any key to return...                %s│%s" "$ESC" "$row" "$start_c" "$COLOR_BORDER" "$COLOR_DIM"
"$COLOR_BORDER" "$COLOR_RESET"; row=$((row+1))
    printf "%s[%d;%dH%s└%s┘%s" "$ESC" "$row" "$start_c" "$COLOR_BORDER" "$hline" "$COLOR_RESET"; row=$((row+1))

    while true; do
        local in
        in=$(dd bs=10 count=1 2>/dev/null || true)
        if [ -n "$in" ]; then break; fi
        sleep 0.05
    done
}

# ------------------------------------------------------------------------------
# Arena & HUD Rendering
# ------------------------------------------------------------------------------
draw_static_arena() {
    printf "\033[2J" # Clear entire screen
    update_term_geometry
    init_theme

    local total_w=$(( BOARD_W * CELL_WIDTH + 2 ))
    local high_score
    high_score=$(get_high_score)

    # 1. Clean Borderless Top HUD Line: Only current score and HI score
    local cur_y=$OFFSET_Y
    local cur_x=$OFFSET_X

    # Width: "SCORE: " (7) + 5 + "             " (13) + "HI: " (4) + 5 = 34 columns
    local hud_line
    hud_line=$(printf "%sSCORE: %s%-5d             %sHI: %s%-5d" \
        "$COLOR_HUD_TXT" "$COLOR_HUD_VAL" 0 \
        "$COLOR_HUD_TXT" "$COLOR_HUD_VAL" "$high_score")

    local hud_col=$(( OFFSET_X + (total_w - 34) / 2 ))
    [ "$hud_col" -lt 1 ] && hud_col=1
    printf "\033[%d;%dH%s%s" "$cur_y" "$hud_col" "$hud_line" "$COLOR_RESET"
    cur_y=$((cur_y + 1))

    # 2. Game Arena Box
    # Top Arena Border
    local arena_tl="┌" arena_tr="┐" arena_bl="└" arena_br="┘" arena_h="─" arena_v="│"
    local arena_top_line=""
    for ((i=0; i<BOARD_W * CELL_WIDTH; i++)); do arena_top_line="${arena_top_line}${arena_h}"; done

    printf "\033[%d;%dH%s%s%s%s%s" "$cur_y" "$cur_x" "$COLOR_BORDER" "$arena_tl" "$arena_top_line" "$arena_tr" "$COLOR_RESET"
    cur_y=$((cur_y + 1))

    # Empty Arena Rows
    local empty_row=""
    for ((i=0; i<BOARD_W * CELL_WIDTH; i++)); do empty_row="${empty_row} "; done
    for ((r=0; r<BOARD_H; r++)); do
        printf "\033[%d;%dH%s%s%s%s%s%s%s" "$cur_y" "$cur_x" "$COLOR_BORDER" "$arena_v" "$COLOR_RESET" "$empty_row" "$COLOR_BORDER" "$arena_v"
"$COLOR_RESET"
        cur_y=$((cur_y + 1))
    done

    # Bottom Arena Border
    printf "\033[%d;%dH%s%s%s%s%s" "$cur_y" "$cur_x" "$COLOR_BORDER" "$arena_bl" "$arena_top_line" "$arena_br" "$COLOR_RESET"
    cur_y=$((cur_y + 1))

    # Controls Helper Footer (only space will pause)
    local hint="Controls: [W A S D / Arrows] Move  |  [Space] Pause  |  [Q] Quit"
    local hint_x=$(( (TERM_COLS - ${#hint}) / 2 ))
    [ "$hint_x" -lt 1 ] && hint_x=1
    printf "\033[%d;%dH%s%s%s" "$cur_y" "$hint_x" "$COLOR_DIM" "$hint" "$COLOR_RESET"
}

update_hud() {
    local score="$1"
    local high_score="$2"

    local cur_y=$OFFSET_Y
    local total_w=$(( BOARD_W * CELL_WIDTH + 2 ))
    local hud_col=$(( OFFSET_X + (total_w - 34) / 2 ))
    [ "$hud_col" -lt 1 ] && hud_col=1

    printf "\033[%d;%dH%sSCORE: %s%-5d             %sHI: %s%-5d%s" \
        "$cur_y" "$hud_col" \
        "$COLOR_HUD_TXT" "$COLOR_HUD_VAL" "$score" \
        "$COLOR_HUD_TXT" "$COLOR_HUD_VAL" "$high_score" \
        "$COLOR_RESET"
}

# ------------------------------------------------------------------------------
# Cell Coordinate Conversions
```

```
# ---------------------------------------------------------------------------
# Arena content starts at row (OFFSET_Y + 2) and col (OFFSET_X + 1)
get_screen_row() {
    local gy="$1"
    echo $(( OFFSET_Y + 2 + gy ))
}

get_screen_col() {
    local gx="$1"
    echo $(( OFFSET_X + 1 + gx * CELL_WIDTH ))
}

# ---------------------------------------------------------------------------
# Pause Dialog
# ---------------------------------------------------------------------------
show_pause_overlay() {
    local box_w=28
    local inner_w=26
    local start_c=$(( (TERM_COLS - box_w) / 2 ))
    local start_r=$(( (TERM_ROWS - 7) / 2 ))

    local hline=""
    for ((i=0; i<inner_w; i++)); do hline="${hline}─"; done

    printf "%s[%d;%dH%s┌%s┐%s" "$ESC" "$start_r" "$start_c" "$COLOR_MODAL" "$hline" "$COLOR_RESET"
    printf "%s[%d;%dH%s│%s       P A U S E D        %s│%s" "$ESC" "$((start_r+1))" "$start_c" "$COLOR_MODAL" "$COLOR_WHITE" "$COLOR_MODAL"
"$COLOR_RESET"
    printf "%s[%d;%dH%s├%s┤%s" "$ESC" "$((start_r+2))" "$start_c" "$COLOR_MODAL" "$hline" "$COLOR_RESET"
    printf "%s[%d;%dH%s│%s   [Space]     Resume     %s│%s" "$ESC" "$((start_r+3))" "$start_c" "$COLOR_MODAL" "$COLOR_HUD_TXT" "$COLOR_MODAL"
"$COLOR_RESET"
    printf "%s[%d;%dH%s│%s   [Q]         Quit       %s│%s" "$ESC" "$((start_r+4))" "$start_c" "$COLOR_MODAL" "$COLOR_HUD_TXT" "$COLOR_MODAL"
"$COLOR_RESET"
    printf "%s[%d;%dH%s└%s┘%s" "$ESC" "$((start_r+5))" "$start_c" "$COLOR_MODAL" "$hline" "$COLOR_RESET"

    while true; do
        local in
        in=$(dd bs=10 count=1 2>/dev/null || true)
        case "$in" in
            " ") break ;;
            q|Q|$'\e') return 1 ;;
        esac
        sleep 0.05
    done

    # 3-2-1 Countdown before resuming
    for num in 3 2 1; do
        printf "%s[%d;%dH%s│%s     Resuming in %d...      %s│%s" "$ESC" "$((start_r+3))" "$start_c" "$COLOR_MODAL" "$COLOR_GOLD" "$num" "$COLOR_MODAL"
"$COLOR_RESET"
        sleep 0.4
    done

    return 0
}

# ---------------------------------------------------------------------------
# Game Over Dialog & Score Recording
# ---------------------------------------------------------------------------
show_game_over() {
    local score="$1"
    local snake_len="$2"
    local foods_eaten="$3"
    local time_sec="$4"
    local high_score="$5"

    local is_new_record=0
    if [ "$score" -gt "$high_score" ]; then
        is_new_record=1
        high_score="$score"
    fi

    # Save high score
    if [ "$score" -gt 0 ]; then
        save_high_score "$score"
    fi

    local min=$(( time_sec / 60 ))
    local sec=$(( time_sec % 60 ))
    local time_fmt
    time_fmt=$(printf "%02d:%02d" "$min" "$sec")

    local inner_w=$(( BOARD_W * CELL_WIDTH ))

    local BG_OVERLAY="${ESC}[48;5;52m"
    local FG_BORDER="${ESC}[1;38;5;196m"
    local FG_TITLE="${ESC}[1;38;5;226m"
    local FG_LABEL="${ESC}[38;5;224m"
    local FG_VAL="${ESC}[1;38;5;220m"
    local FG_TXT="${ESC}[1;37m"
    local FG_BTN="${ESC}[1;38;5;203m"

    render_game_over_overlay() {
        local title="G A M E   O V E R"
        if [ "$is_new_record" -eq 1 ]; then
            title="NEW HIGH SCORE!"
        fi
        local t_len=${#title}
        local t_pad=$(( inner_w - t_len ))
        local t_pad_l=$(( t_pad / 2 ))
```

```
        local t_pad_r=$(( t_pad - t_pad_l ))

        # Top border in bright red
        printf "%s[%d;%dH%s┌" "$ESC" "$((OFFSET_Y + 1))" "$OFFSET_X" "$FG_BORDER"
        for ((i=0; i<inner_w; i++)); do printf "─"; done
        printf "┐%s\n" "$COLOR_RESET"

        for ((r=0; r<BOARD_H; r++)); do
            local cur_y=$(( OFFSET_Y + 2 + r ))
            printf "%s[%d;%dH%s│%s" "$ESC" "$cur_y" "$OFFSET_X" "$FG_BORDER" "$COLOR_RESET"
            printf "%s" "$BG_OVERLAY"

            case "$r" in
                0|1)
                    printf "%64s" ""
                    ;;
                2)
                    printf "%*s%s%s%s%s%*s" "$t_pad_l" "" "$FG_TITLE" "$title" "$COLOR_RESET$BG_OVERLAY" "$t_pad_r" ""
                    ;;
                3)
                    printf "  "
                    for ((d=0; d<60; d++)); do printf "─"; done
                    printf "  "
                    ;;
                4)
                    printf "%64s" ""
                    ;;
                5)
                    printf "                    %sFinal Score   : %s%-10d%s                    " "$FG_LABEL" "$FG_VAL" "$score" "$COLOR_RESET$BG_OVERLAY"
                    ;;
                6)
                    printf "                    %sHigh Score    : %s%-10d%s                    " "$FG_LABEL" "$FG_VAL" "$high_score"
"$COLOR_RESET$BG_OVERLAY"
                    ;;
                7)
                    printf "                    %sSnake Length  : %s%-10d%s                    " "$FG_LABEL" "$FG_TXT" "$snake_len"
"$COLOR_RESET$BG_OVERLAY"
                    ;;
                8)
                    printf "                    %sFoods Eaten   : %s%-10d%s                    " "$FG_LABEL" "$FG_TXT" "$foods_eaten"
"$COLOR_RESET$BG_OVERLAY"
                    ;;
                9)
                    printf "                    %sTime Survived : %s%-10s%s                    " "$FG_LABEL" "$FG_TXT" "$time_fmt"
"$COLOR_RESET$BG_OVERLAY"
                    ;;
                10)
                    printf "%64s" ""
                    ;;
                11)
                    printf "  "
                    for ((d=0; d<60; d++)); do printf "─"; done
                    printf "  "
                    ;;
                12)
                    printf "%64s" ""
                    ;;
                13)
                    printf "               %s[R]%s Play Again       %s[M]%s Main Menu%*s" "$FG_BTN" "$FG_TXT" "$FG_BTN" "$FG_TXT" 15 ""
                    ;;
                14)
                    printf "               %s[Q]%s Quit Game%*s" "$FG_BTN" "$FG_TXT" 36 ""
                    ;;
                15|16|17)
                    printf "%64s" ""
                    ;;
            esac
            printf "%s%s│%s" "$COLOR_RESET" "$FG_BORDER" "$COLOR_RESET"
        done

        # Bottom border in bright red
        printf "%s[%d;%dH%s└" "$ESC" "$((OFFSET_Y + 2 + BOARD_H))" "$OFFSET_X" "$FG_BORDER"
        for ((i=0; i<inner_w; i++)); do printf "─"; done
        printf "┘%s" "$COLOR_RESET"
    }

    render_game_over_overlay

    while true; do
        local in
        in=$(dd bs=10 count=1 2>/dev/null || true)
        case "$in" in
            r|R) return 0 ;; # Restart
            m|M) return 2 ;; # Main menu
            q|Q|$'\e') return 3 ;; # Quit
        esac
        sleep 0.05
    done
}

# ------------------------------------------------------------------------------
# Core Game Loop
# ------------------------------------------------------------------------------
run_game() {
    init_theme
    draw_static_arena

    local high_score
```

```
high_score=$(get_high_score)

# Snake state (dynamically centered on board)
local cx=$(( BOARD_W / 2 ))
local cy=$(( BOARD_H / 2 ))
local snake_len=3
local snake_x=( $((cx)) $((cx - 1)) $((cx - 2)) )
local snake_y=( $((cy)) $((cy)) $((cy)) )

# Occupancy tracking: 1D array indexed by y * BOARD_W + x
local grid=()
local total_cells=$(( BOARD_W * BOARD_H ))
for ((i=0; i<total_cells; i++)); do grid[i]=0; done
grid[$(( cy * BOARD_W + cx ))]=1
grid[$(( cy * BOARD_W + cx - 1 ))]=1
grid[$(( cy * BOARD_W + cx - 2 ))]=1

# Direction: UP, DOWN, LEFT, RIGHT
local dir="RIGHT"
local next_dir="RIGHT"

# Score & Counters
local score=0
local foods_eaten=0
local start_time
start_time=$(date +%s)

# Base tick speed based on difficulty
local base_delay=0.08
case "$DIFFICULTY" in
    0) base_delay=0.12 ;;
    1) base_delay=0.08 ;;
    2) base_delay=0.05 ;;
    3) base_delay=0.035 ;;
esac
local current_delay="$base_delay"

# Spawning food helper
local food_x=0
local food_y=0
spawn_food() {
    while true; do
        food_x=$(( RANDOM % BOARD_W ))
        food_y=$(( RANDOM % BOARD_H ))
        local idx=$(( food_y * BOARD_W + food_x ))
        if [ "${grid[idx]:-0}" -eq 0 ]; then
            break
        fi
    done
    local s_row
    s_row=$(get_screen_row "$food_y")
    local s_col
    s_col=$(get_screen_col "$food_x")
    printf "\033[%d;%dH%s◆ %s" "$s_row" "$s_col" "$COLOR_FOOD" "$COLOR_RESET"
}

# Initial draw of snake
for ((i=1; i<snake_len; i++)); do
    local r
    r=$(get_screen_row "${snake_y[i]}")
    local c
    c=$(get_screen_col "${snake_x[i]}")
    printf "\033[%d;%dH%s● %s" "$r" "$c" "$COLOR_BODY" "$COLOR_RESET"
done
local hr
hr=$(get_screen_row "${snake_y[0]}")
local hc
hc=$(get_screen_col "${snake_x[0]}")
printf "\033[%d;%dH%s► %s" "$hr" "$hc" "$COLOR_HEAD" "$COLOR_RESET"

spawn_food

# Main active play loop
while true; do
    # 1. Non-blocking input reading
    local raw_input
    raw_input=$(dd bs=10 count=1 2>/dev/null || true)

    if [ -n "$raw_input" ]; then
        # Check arrow keys first
        case "$raw_input" in
            *"[A"*|*"OA"*) [ "$dir" != "DOWN" ] && next_dir="UP" ;;
            *"[B"*|*"OB"*) [ "$dir" != "UP" ] && next_dir="DOWN" ;;
            *"[C"*|*"OC"*) [ "$dir" != "LEFT" ] && next_dir="RIGHT" ;;
            *"[D"*|*"OD"*) [ "$dir" != "RIGHT" ] && next_dir="LEFT" ;;
            *[wWkK]*) [ "$dir" != "DOWN" ] && next_dir="UP" ;;
            *[sSjJ]*) [ "$dir" != "UP" ] && next_dir="DOWN" ;;
            *[aAhH]*) [ "$dir" != "RIGHT" ] && next_dir="LEFT" ;;
            *[dDlL]*) [ "$dir" != "LEFT" ] && next_dir="RIGHT" ;;
            " "*)
                show_pause_overlay || return 2
                # Redraw arena after unpausing
                draw_static_arena
                # Redraw food
                local fr fc
                fr=$(get_screen_row "$food_y")
                fc=$(get_screen_col "$food_x")
                printf "\033[%d;%dH%s◆ %s" "$fr" "$fc" "$COLOR_FOOD" "$COLOR_RESET"
```

```
            # Redraw snake body
            for ((i=1; i<snake_len; i++)); do
                local sr sc
                sr=$(get_screen_row "${snake_y[i]}")
                sc=$(get_screen_col "${snake_x[i]}")
                printf "\033[%d;%dH%s● %s" "$sr" "$sc" "$COLOR_BODY" "$COLOR_RESET"
            done
            # Redraw snake head
            local shr shc
            shr=$(get_screen_row "${snake_y[0]}")
            shc=$(get_screen_col "${snake_x[0]}")
            printf "\033[%d;%dH%s%s%s" "$shr" "$shc" "$COLOR_HEAD" "$head_symbol" "$COLOR_RESET"
            update_hud "$score" "$high_score"
            ;;
        *[qQ]*|$'\e'*)
            return 2 # Back to menu
            ;;
    esac
fi

dir="$next_dir"

# 2. Compute new head position
local cur_hx="${snake_x[0]}"
local cur_hy="${snake_y[0]}"
local new_hx="$cur_hx"
local new_hy="$cur_hy"
local head_symbol="● "

case "$dir" in
    UP)
        new_hy=$((cur_hy - 1))
        head_symbol="▲ "
        ;;
    DOWN)
        new_hy=$((cur_hy + 1))
        head_symbol="▼ "
        ;;
    LEFT)
        new_hx=$((cur_hx - 1))
        head_symbol="◄ "
        ;;
    RIGHT)
        new_hx=$((cur_hx + 1))
        head_symbol="► "
        ;;
esac

# 3. Wall collision or Wrap-around
if [ "$WALL_MODE" -eq 1 ]; then
    # Solid walls: collision kills
    if [ "$new_hx" -lt 0 ] || [ "$new_hx" -ge "$BOARD_W" ] || \
       [ "$new_hy" -lt 0 ] || [ "$new_hy" -ge "$BOARD_H" ]; then
        # Crash!
        break
    fi
else
    # Wrap-around portal mode
    if [ "$new_hx" -lt 0 ]; then new_hx=$((BOARD_W - 1)); fi
    if [ "$new_hx" -ge "$BOARD_W" ]; then new_hx=0; fi
    if [ "$new_hy" -lt 0 ]; then new_hy=$((BOARD_H - 1)); fi
    if [ "$new_hy" -ge "$BOARD_H" ]; then new_hy=0; fi
fi

# 4. Check self collision
local new_idx=$(( new_hy * BOARD_W + new_hx ))
local tail_idx=$(( snake_y[snake_len - 1] * BOARD_W + snake_x[snake_len - 1] ))

# Moving into current tail is permitted if tail will vacate this step
if [ "${grid[new_idx]:-0}" -eq 1 ] && [ "$new_idx" -ne "$tail_idx" ]; then
    # Self bite! Crash!
    break
fi

# 5. Check food collision
local ate_food=0

if [ "$new_hx" -eq "$food_x" ] && [ "$new_hy" -eq "$food_y" ]; then
    ate_food=1
    score=$(( score + 10 ))
    foods_eaten=$(( foods_eaten + 1 ))
fi

# 6. Update Snake Body and Rendering
# Replace former head with body character
local old_head_r=$(( OFFSET_Y + 2 + cur_hy ))
local old_head_c=$(( OFFSET_X + 1 + cur_hx * CELL_WIDTH ))
printf "\033[%d;%dH%s● %s" "$old_head_r" "$old_head_c" "$COLOR_BODY" "$COLOR_RESET"

if [ "$ate_food" -eq 1 ]; then
    # Snake grows: prepend new head, do NOT erase tail
    snake_x=("$new_hx" "${snake_x[@]}")
    snake_y=("$new_hy" "${snake_y[@]}")
    snake_len=$(( snake_len + 1 ))
    grid[new_idx]=1
    spawn_food

    # Recalculate dynamic speed only when food is eaten
```

```
            if [ "$score" -gt 100 ] && [ "$DIFFICULTY" -le 1 ]; then
                current_delay=$(awk -v b="$base_delay" -v sc="$score" 'BEGIN { d = b - (sc/1500)*0.02; if(d < 0.04) d=0.04; print d }' 2>/dev/null ||
echo "$base_delay")
            fi
        else
            # Erase tail
            local last_tail_x="${snake_x[snake_len - 1]}"
            local last_tail_y="${snake_y[snake_len - 1]}"
            local old_tail_idx=$(( last_tail_y * BOARD_W + last_tail_x ))
            grid[old_tail_idx]=0

            local tail_r=$(( OFFSET_Y + 2 + last_tail_y ))
            local tail_c=$(( OFFSET_X + 1 + last_tail_x * CELL_WIDTH ))
            printf "\033[%d;%dH  " "$tail_r" "$tail_c"

            # Shift snake body array
            snake_x=("$new_hx" "${snake_x[@]:0:$((snake_len - 1))}")
            snake_y=("$new_hy" "${snake_y[@]:0:$((snake_len - 1))}")
            grid[new_idx]=1
        fi

        # Draw new head
        local new_head_r=$(( OFFSET_Y + 2 + new_hy ))
        local new_head_c=$(( OFFSET_X + 1 + new_hx * CELL_WIDTH ))
        printf "\033[%d;%dH%s%s%s" "$new_head_r" "$new_head_c" "$COLOR_HEAD" "$head_symbol" "$COLOR_RESET"

        # Update HUD
        update_hud "$score" "$high_score"

        sleep "$current_delay"
    done

    # Game ended: flash head red
    local crash_r=$(( OFFSET_Y + 2 + snake_y[0] ))
    local crash_c=$(( OFFSET_X + 1 + snake_x[0] * CELL_WIDTH ))
    printf "\033[%d;%dH%s💥%s" "$crash_r" "$crash_c" "$COLOR_RED" "$COLOR_RESET"
    sleep 0.4

    local now_time
    now_time=$(date +%s)
    local alive_sec=$(( now_time - start_time ))

    # Show game over dialog
    show_game_over "$score" "$snake_len" "$foods_eaten" "$alive_sec" "$high_score"
    return $?
}

# ------------------------------------------------------------------------------
# Main Menu & Settings
# ------------------------------------------------------------------------------
show_main_menu() {
    while true; do
        printf "\033[2J"
        update_term_geometry
        init_theme

        local box_w=56
        local inner_w=54
        local start_c=$(( (TERM_COLS - box_w) / 2 ))
        [ "$start_c" -lt 1 ] && start_c=1
        local logo_c=$(( (TERM_COLS - 48) / 2 ))
        [ "$logo_c" -lt 1 ] && logo_c=1
        local row=$(( (TERM_ROWS - 17) / 2 ))
        [ "$row" -lt 1 ] && row=1

        # Retro Arcade ASCII Logo (Centered, subtitle removed)
        printf "%s[%d;%dH%s  ▄▄▄
```

```
                "$COLOR_BORDER" "$COLOR_RESET" \
                "$COLOR_DIM" "$COLOR_WHITE" "$key" "$COLOR_DIM" "$COLOR_RESET" \
                "$COLOR_HUD_TXT" "$padded" \
                "$COLOR_BORDER" "$COLOR_RESET"
            row=$((row+1))
        }

        render_menu_item "1" "START GAME"
        render_menu_item "2" "DIFFICULTY   : $diff_name"
        render_menu_item "3" "THEME        : $THEME_NAME"
        render_menu_item "4" "WALLS        : $wall_name"
        render_menu_item "5" "HOW TO PLAY & CONTROLS"
        render_menu_item "Q" "QUIT"

        printf "%s[%d;%dH%s└%s┘%s" "$ESC" "$row" "$start_c" "$COLOR_BORDER" "$hline" "$COLOR_RESET"; row=$((row+2))

        local hint="Press [1-5] or [Q] to choose an option..."
        local hint_c=$(( (TERM_COLS - ${#hint}) / 2 ))
        [ "$hint_c" -lt 1 ] && hint_c=1
        printf "%s[%d;%dH%s%s%s" "$ESC" "$row" "$hint_c" "$COLOR_DIM" "$hint" "$COLOR_RESET"

        # Wait for menu input
        while true; do
            local choice
            choice=$(dd bs=10 count=1 2>/dev/null || true)
            case "$choice" in
                1|$'\n'|" ")
                    # Play game
                    while true; do
                        run_game
                        local status=$?
                        if [ "$status" -eq 0 ]; then
                            continue # Restart
                        elif [ "$status" -eq 2 ]; then
                            break # Main menu
                        elif [ "$status" -eq 3 ]; then
                            cleanup # Quit
                        fi
                    done
                    break
                    ;;
                2)
                    DIFFICULTY=$(( (DIFFICULTY + 1) % 4 ))
                    break
                    ;;
                3)
                    THEME=$(( (THEME + 1) % 5 ))
                    init_theme
                    break
                    ;;
                4)
                    WALL_MODE=$(( 1 - WALL_MODE ))
                    break
                    ;;
                5)
                    show_instructions
                    break
                    ;;
                q|Q|$'\e')
                    cleanup
                    ;;
            esac
            sleep 0.05
        done
    done
}

# ------------------------------------------------------------------------------
# Entrypoint
# ------------------------------------------------------------------------------
# CLI Flag support
while [ $# -gt 0 ]; do
    case "$1" in
        -h|--help)
            echo "Usage: ./snake.sh [OPTIONS]"
            echo ""
            echo "Modern Cross-Platform Terminal Snake Game in Bash"
            echo ""
            echo "Options:"
            echo "  -d, --difficulty <0-3>  Difficulty (0=Casual, 1=Normal, 2=Hard, 3=Insane)"
            echo "  -t, --theme <0-4>       Theme (0=Emerald, 1=Cyberpunk, 2=Synthwave, 3=Ice Blue, 4=Mono)"
            echo "  -w, --wrap              Enable wrap-around portal walls"
            echo "  -h, --help              Show this help message and exit"
            exit 0
            ;;
        -d|--difficulty) shift; [ -n "${1:-}" ] && DIFFICULTY="$1" ;;
        -t|--theme) shift; [ -n "${1:-}" ] && THEME="$1" ;;
        -w|--wrap) WALL_MODE=0 ;;
    esac
    shift
done

show_main_menu
```

Project 2: Running ASCII Clock

Problem statement: Write an executable Bash script named clock.sh that repeatedly displays the current time in centered ASCII-art digits. The time and date are obtained using the Unix date command, and the display updates at regular intervals.

Objective

To create a terminal clock that renders the current time as large ASCII digits, positions the clock in the center of the terminal, refreshes it periodically, and responds to keyboard controls.

Features and controls

- Displays the current date and time using the system date command.
- Supports 12-hour and 24-hour time formats, with optional seconds and date display.
- Provides selectable digit styles, color themes, and border styles.
- Updates the display at regular intervals and can blink the time separators.
- Keyboard controls include Q to quit, T to switch time format, S to toggle seconds, D to cycle date display, C to change theme, F to change font, B to change border, and H to toggle help.

How to execute

```
chmod +x clock.sh
./clock.sh
```

Program output

The following screenshot was captured from the clock script while it was running in an interactive terminal instance. The large ASCII digits show the live time, with the date above and controls at the bottom.

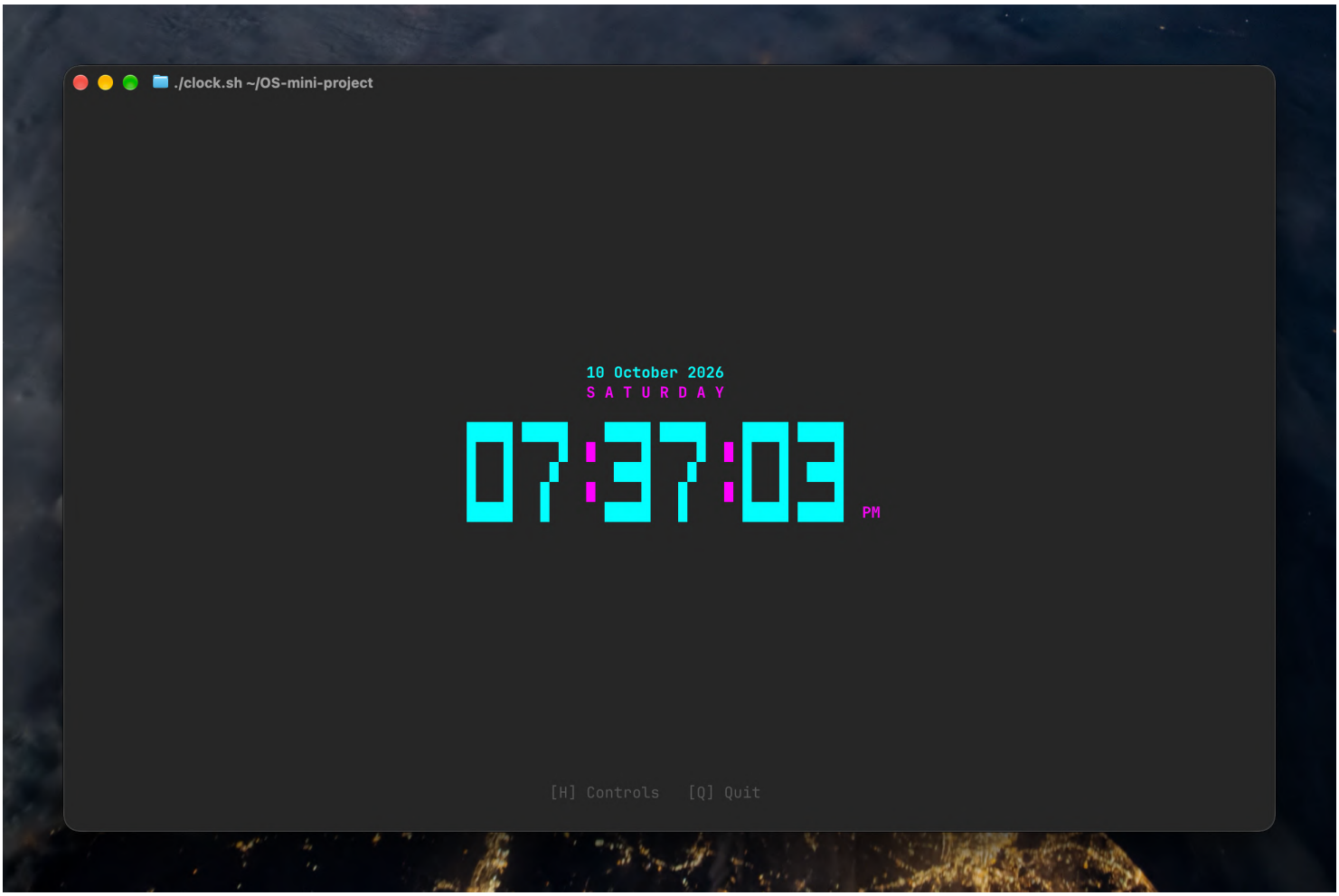


Figure 4. Terminal output of Project 2, running ASCII clock.

Project 2 Source Code: clock.sh

```
#!/usr/bin/env bash

# Ensure UTF-8 locale for character handling across platforms
export LC_ALL="${LC_ALL:-en_US.UTF-8}"
export LANG="${LANG:-en_US.UTF-8}"

# Single-byte ASCII Escape code for ANSI terminal formatting (compatible with all shells and printf %s)
ESC=$(printf '\033')

# Ensure clean exit and terminal restoration
cleanup() {
    printf "%s[?25h" "$ESC"          # Show cursor
    printf "%s[0m" "$ESC"            # Reset colors
    printf "%s[2J%s[H" "$ESC" "$ESC" # Clear screen and move home
    if [ -n "$ORIG_STTY" ]; then
        stty "$ORIG_STTY" 2>/dev/null || true
    fi
    exit 0
}

trap cleanup INT TERM EXIT

# Save original terminal settings and configure non-blocking input
ORIG_STTY=$(stty -g 2>/dev/null || true)
stty -echo -icanon min 0 time 0 2>/dev/null || true

# Hide cursor
printf "%s[?25l" "$ESC"

# Options and state
FORMAT_12H=1          # 1 = 12-hour with AM/PM, 0 = 24-hour (no 24H text)
SHOW_SECONDS=1        # 1 = Show HH:MM:SS, 0 = Show HH:MM
SHOW_DATE=1           # 1 = Large 2-Line Day & Date, 2 = Compact, 0 = Hidden
BLINK_COLON=1         # 1 = Blink colons every interval
FONT_STYLE=1          # 1 = Modern Bold Block (Default), 0 = Classic 7-Segment, 2 = Tech Slender
COLOR_THEME=1         # 1 = Cyberpunk Neon (Default), 0 = Emerald, 2 = Amber, 3 = Solar, 4 = Violet, 5 = Mono
BORDER_STYLE=0        # 0 = No border (Clean Modern Default), 1 = Double (╔═══╗), 2 = Rounded (╭───╮), 3 = Dashed (+---+), 4 = Solid (┌───┐)
SHOW_HELP=0           # 0 = Hidden, 1 = Show hotkey bar at bottom

# Parse command line options
while [ $# -gt 0 ]; do
    case "$1" in
        -h|--help)
            echo "Usage: ./clock.sh [OPTIONS]"
            echo ""
            echo "Modern ASCII Running Digital Clock in Bash"
            echo ""
            echo "Options:"
            echo "  -12, --12h            Use 12-hour format with AM/PM (default)"
            echo "  -24, --24h            Use 24-hour format (clean without 24H label)"
            echo "  -s, --no-seconds      Hide seconds"
            echo "  -d, --no-date         Hide date header"
            echo "  -b, --border <0-4>    Border style (0=None, 1=Double, 2=Rounded, 3=Dashed, 4=Solid)"
            echo "  --no-blink            Disable blinking colons"
            echo "  -c, --theme <0-5>     Color theme (1=Cyberpunk, 0=Emerald, 2=Amber, 3=Solar, 4=Violet, 5=Mono)"
            echo "  -f, --font <0-2>      Font style (1=Modern Bold Block, 0=Classic 7-Segment, 2=Tech Slender)"
            echo "  -h, --help            Show this help message and exit"
            echo ""
            echo "Keyboard Controls while running:"
            echo "  q, ESC   : Quit clock"
            echo "  h, ?     : Toggle controls / Return to homescreen"
            echo "  t        : Toggle 12h / 24h mode"
            echo "  s        : Toggle seconds display"
            echo "  d        : Toggle date display (Large / Compact / Off)"
            echo "  c        : Cycle color themes"
            echo "  f        : Cycle font styles"
            echo "  b        : Cycle border styles (None / Double / Rounded / Dashed / Solid)"
            exit 0
            ;;
        -12|--12h) FORMAT_12H=1 ;;
        -24|--24h) FORMAT_12H=0 ;;
        -s|--no-seconds) SHOW_SECONDS=0 ;;
        -d|--no-date) SHOW_DATE=0 ;;
        -b|--border)
            shift
            if [ -n "${1:-}" ]; then BORDER_STYLE="$1"; fi
            ;;
        --no-blink) BLINK_COLON=0 ;;
        -c|--theme)
            shift
            if [ -n "${1:-}" ]; then COLOR_THEME="$1"; fi
            ;;
        -f|--font)
            shift
            if [ -n "${1:-}" ]; then FONT_STYLE="$1"; fi
            ;;
    esac
    shift
done

update_term_size() {
    local size
    size=$(stty size 2>/dev/null || echo "24 80")
    read -r TERM_ROWS TERM_COLS <<< "$size"
```

```
    [ -z "$TERM_ROWS" ] || [ "$TERM_ROWS" -le 0 ] && TERM_ROWS=24
    [ -z "$TERM_COLS" ] || [ "$TERM_COLS" -le 0 ] && TERM_COLS=80
    printf "%s[2J" "$ESC"  # Clear screen on resize
}

trap update_term_size WINCH

# ------------------------------------------------------------------------------
# Color Palettes
# ------------------------------------------------------------------------------
get_theme_colors() {
    case "$COLOR_THEME" in
        0) # Emerald Matrix Green
            COLOR_BORDER="${ESC}[38;5;46m"
            COLOR_DIGIT="${ESC}[1;38;5;48m"
            COLOR_COLON="${ESC}[1;38;5;120m"
            COLOR_DATE="${ESC}[1;38;5;46m"
            COLOR_DATE_SUB="${ESC}[1;38;5;157m"
            COLOR_AMPM="${ESC}[1;38;5;119m"
            THEME_NAME="Emerald Matrix"
            ;;
        1) # Cyberpunk Neon
            COLOR_BORDER="${ESC}[38;5;51m"
            COLOR_DIGIT="${ESC}[1;38;5;51m"
            COLOR_COLON="${ESC}[1;38;5;201m"
            COLOR_DATE="${ESC}[1;38;5;201m"
            COLOR_DATE_SUB="${ESC}[1;38;5;51m"
            COLOR_AMPM="${ESC}[1;38;5;201m"
            THEME_NAME="Cyberpunk Neon"
            ;;
        2) # Amber CRT Terminal
            COLOR_BORDER="${ESC}[38;5;214m"
            COLOR_DIGIT="${ESC}[1;38;5;220m"
            COLOR_COLON="${ESC}[1;38;5;208m"
            COLOR_DATE="${ESC}[1;38;5;214m"
            COLOR_DATE_SUB="${ESC}[1;38;5;222m"
            COLOR_AMPM="${ESC}[1;38;5;214m"
            THEME_NAME="Amber CRT"
            ;;
        3) # Solarized Gold
            COLOR_BORDER="${ESC}[38;5;178m"
            COLOR_DIGIT="${ESC}[1;38;5;221m"
            COLOR_COLON="${ESC}[1;38;5;166m"
            COLOR_DATE="${ESC}[1;38;5;220m"
            COLOR_DATE_SUB="${ESC}[1;38;5;229m"
            COLOR_AMPM="${ESC}[1;38;5;178m"
            THEME_NAME="Solar Gold"
            ;;
        4) # Electric Violet
            COLOR_BORDER="${ESC}[38;5;141m"
            COLOR_DIGIT="${ESC}[1;38;5;183m"
            COLOR_COLON="${ESC}[1;38;5;199m"
            COLOR_DATE="${ESC}[1;38;5;199m"
            COLOR_DATE_SUB="${ESC}[1;38;5;141m"
            COLOR_AMPM="${ESC}[1;38;5;141m"
            THEME_NAME="Electric Violet"
            ;;
        5) # Sleek Monochrome
            COLOR_BORDER="${ESC}[38;5;248m"
            COLOR_DIGIT="${ESC}[1;38;5;255m"
            COLOR_COLON="${ESC}[1;38;5;250m"
            COLOR_DATE="${ESC}[1;38;5;255m"
            COLOR_DATE_SUB="${ESC}[1;38;5;248m"
            COLOR_AMPM="${ESC}[1;38;5;245m"
            THEME_NAME="Pure Monochrome"
            ;;
    esac
    COLOR_RESET="${ESC}[0m"
    COLOR_DIM="${ESC}[38;5;240m"
}

# ------------------------------------------------------------------------------
# ASCII Glyphs Definition
# ------------------------------------------------------------------------------

# Style 0: Classic 3-Line 7-Segment (True to Assignment Reference)
# Digits are 3 chars wide, 3 lines tall
get_classic_glyph() {
    local char="$1"
    local line="$2"
    local show_colon="$3"
    case "$char" in
        0) [ "$line" -eq 1 ] && echo " _ " || { [ "$line" -eq 2 ] && echo "| |" || echo "|_|"; } ;;
        1) [ "$line" -eq 1 ] && echo "   " || { [ "$line" -eq 2 ] && echo "  |" || echo "  |"; } ;;
        2) [ "$line" -eq 1 ] && echo " _ " || { [ "$line" -eq 2 ] && echo " _|" || echo "|_ "; } ;;
        3) [ "$line" -eq 1 ] && echo " _ " || { [ "$line" -eq 2 ] && echo " _|" || echo " _|"; } ;;
        4) [ "$line" -eq 1 ] && echo "   " || { [ "$line" -eq 2 ] && echo "|_|" || echo "  |"; } ;;
        5) [ "$line" -eq 1 ] && echo " _ " || { [ "$line" -eq 2 ] && echo "|_ " || echo " _|"; } ;;
        6) [ "$line" -eq 1 ] && echo " _ " || { [ "$line" -eq 2 ] && echo "|_ " || echo "|_|"; } ;;
        7) [ "$line" -eq 1 ] && echo " _ " || { [ "$line" -eq 2 ] && echo "  |" || echo "  |"; } ;;
        8) [ "$line" -eq 1 ] && echo " _ " || { [ "$line" -eq 2 ] && echo "|_|" || echo "|_|"; } ;;
        9) [ "$line" -eq 1 ] && echo " _ " || { [ "$line" -eq 2 ] && echo "|_|" || echo " _|"; } ;;
        :)
            if [ "$show_colon" -eq 1 ]; then
                [ "$line" -eq 1 ] && echo " " || { [ "$line" -eq 2 ] && echo ":" || echo "."; }
            else
                echo " "
            fi
```

```
            ;;
        " ") echo " " ;;
        *) echo "   " ;;
    esac
}

# Style 1: Modern Bold Block (5 lines tall)
get_block_glyph() {
    local char="$1"
    local line="$2"
    local show_colon="$3"
    case "$char" in
        0)
            case "$line" in
                1) echo "████" ;;
                2) echo "█  █" ;;
                3) echo "█  █" ;;
                4) echo "█  █" ;;
                5) echo "████" ;;
            esac ;;
        1)
            case "$line" in
                1) echo "  ▐█ " ;;
                2) echo " ▐ ▐ " ;;
                3) echo "   ▐ " ;;
                4) echo "   ▐ " ;;
                5) echo "  ▐▐▐" ;;
            esac ;;
        2)
            case "$line" in
                1) echo "▐▀▀▌" ;;
                2) echo "   ▐" ;;
                3) echo "▐▀▀▌" ;;
                4) echo "▐   " ;;
                5) echo "▐▄▄▌" ;;
            esac ;;
        3)
            case "$line" in
                1) echo "▐▀▀▌" ;;
                2) echo "   ▐" ;;
                3) echo " ▀▀▌" ;;
                4) echo "   ▐" ;;
                5) echo "▐▄▄▌" ;;
            esac ;;
        4)
            case "$line" in
                1) echo "▐  ▐" ;;
                2) echo "▐  ▐" ;;
                3) echo "▐▄▄▌" ;;
                4) echo "   ▐" ;;
                5) echo "   ▐" ;;
            esac ;;
        5)
            case "$line" in
                1) echo "▐▀▀▀" ;;
                2) echo "▐   " ;;
                3) echo "▐▀▀▌" ;;
                4) echo "   ▐" ;;
                5) echo "▐▄▄▌" ;;
            esac ;;
        6)
            case "$line" in
                1) echo "▐▀▀▀" ;;
                2) echo "▐   " ;;
                3) echo "▐▀▀▌" ;;
                4) echo "▐  ▐" ;;
                5) echo "▐▄▄▌" ;;
            esac ;;
        7)
            case "$line" in
                1) echo "▐▀▀▌" ;;
                2) echo "   ▐" ;;
                3) echo "  ▐ " ;;
                4) echo "  ▐  " ;;
                5) echo "  ▐  " ;;
            esac ;;
        8)
            case "$line" in
                1) echo "▐▀▀▌" ;;
                2) echo "▐  ▐" ;;
                3) echo "▐▀▀▌" ;;
                4) echo "▐  ▐" ;;
                5) echo "▐▄▄▌" ;;
            esac ;;
        9)
            case "$line" in
                1) echo "▐▀▀▌" ;;
                2) echo "▐  ▐" ;;
                3) echo "▐▀▀▌" ;;
                4) echo "   ▐" ;;
                5) echo "▐▄▄▌" ;;
            esac ;;
        :)
            if [ "$show_colon" -eq 1 ]; then
                case "$line" in
                    1|3|5) echo " " ;;
                    2|4)   echo "▐" ;;
                esac
```

```
            else
                echo " "
            fi ;;
        " ") echo "  " ;;
        *) echo "     " ;;
    esac
}

# Style 2: Tech Slender (3 lines tall)
get_tech_glyph() {
    local char="$1"
    local line="$2"
    local show_colon="$3"
    case "$char" in
        0) [ "$line" -eq 1 ] && echo "┌┐" || { [ "$line" -eq 2 ] && echo "│ │" || echo "└┘"; } ;;
        1) [ "$line" -eq 1 ] && echo " ┐ " || { [ "$line" -eq 2 ] && echo "  │ " || echo " ┴ "; } ;;
        2) [ "$line" -eq 1 ] && echo "╶┐" || { [ "$line" -eq 2 ] && echo "┌─┘" || echo "└─"; } ;;
        3) [ "$line" -eq 1 ] && echo "╶┐" || { [ "$line" -eq 2 ] && echo " ─┤" || echo "╶┘"; } ;;
        4) [ "$line" -eq 1 ] && echo "╷ ╷" || { [ "$line" -eq 2 ] && echo "└─┤" || echo "  ╵"; } ;;
        5) [ "$line" -eq 1 ] && echo "┌─" || { [ "$line" -eq 2 ] && echo "└─┐" || echo "─┘"; } ;;
        6) [ "$line" -eq 1 ] && echo "┌─" || { [ "$line" -eq 2 ] && echo "├─┐" || echo "└┘"; } ;;
        7) [ "$line" -eq 1 ] && echo "╶┐" || { [ "$line" -eq 2 ] && echo "  ┤" || echo "  ╵"; } ;;
        8) [ "$line" -eq 1 ] && echo "┌┐" || { [ "$line" -eq 2 ] && echo "├┤" || echo "└┘"; } ;;
        9) [ "$line" -eq 1 ] && echo "┌┐" || { [ "$line" -eq 2 ] && echo "└┤" || echo "╶┘"; } ;;
        :)
            if [ "$show_colon" -eq 1 ]; then
                [ "$line" -eq 1 ] && echo " " || { [ "$line" -eq 2 ] && echo "•" || echo "•"; }
            else
                echo " "
            fi ;;
        " ") echo " " ;;
        *) echo "   " ;;
    esac
}

# Get line of glyph based on current font style
get_glyph_line() {
    local char="$1"
    local line="$2"
    local show_colon="$3"
    case "$FONT_STYLE" in
        0) get_classic_glyph "$char" "$line" "$show_colon" ;;
        1) get_block_glyph "$char" "$line" "$show_colon" ;;
        2) get_tech_glyph "$char" "$line" "$show_colon" ;;
    esac
}

get_glyph_height() {
    case "$FONT_STYLE" in
        0) echo 3 ;;
        1) echo 5 ;;
        2) echo 3 ;;
    esac
}

# ------------------------------------------------------------------------------
# Render Frame
# ------------------------------------------------------------------------------
render_clock() {
    local blink_state="$1"

    get_theme_colors
    local height
    height=$(get_glyph_height)

    # 1. Fetch current time and date in a single Unix date call
    local time_24 time_12 ampm_val day_name date_val
    IFS="|" read -r time_24 time_12 ampm_val day_name date_val <<< "$(date "+%H:%M:%S|%I:%M:%S|%p|%A|%d %B %Y")"

    local time_str ampm_str=""
    if [ "$FORMAT_12H" -eq 1 ]; then
        if [ "$SHOW_SECONDS" -eq 1 ]; then
            time_str="$time_12"
        else
            time_str="${time_12:0:5}"
        fi
        ampm_str="$ampm_val"
    else
        if [ "$SHOW_SECONDS" -eq 1 ]; then
            time_str="$time_24"
        else
            time_str="${time_24:0:5}"
        fi
    fi

    # 2. Build multi-line ASCII output
    local lines=()
    local colon_visible=1
    if [ "$BLINK_COLON" -eq 1 ] && [ "$blink_state" -eq 0 ]; then
        colon_visible=0
    fi

    for ((l=1; l<=height; l++)); do
        local current_row=""
        local len=${#time_str}
        for ((i=0; i<len; i++)); do
            local ch="${time_str:$i:1}"
            local g
```

```
        g=$(get_glyph_line "$ch" "$l" "$colon_visible")
        if [ "$ch" = ":" ]; then
            current_row="${current_row} ${COLOR_COLON}${g}${COLOR_DIGIT} "
        else
            current_row="${current_row}${g} "
        fi
    done
    lines[l]="${current_row% }"
done

# Calculate exact display column width of clock digits/colons
local glyph_w=3
[ "$FONT_STYLE" -eq 1 ] && glyph_w=5

local inner_width=0
for ((i=0; i<${#time_str}; i++)); do
    local ch="${time_str:$i:1}"
    if [ "$ch" = ":" ]; then
        inner_width=$(( inner_width + 3 ))
    elif [ "$ch" = " " ]; then
        [ "$FONT_STYLE" -eq 1 ] && inner_width=$(( inner_width + 3 )) || inner_width=$(( inner_width + 2 ))
    else
        inner_width=$(( inner_width + glyph_w + 1 ))
    fi
done
inner_width=$(( inner_width - 1 ))

# Determine border characters
local b_tl="" b_tr="" b_bl="" b_br="" b_h="" b_v=""
case "$BORDER_STYLE" in
    1) # Double Lined
        b_tl="╔" b_tr="╗" b_bl="╚" b_br="╝" b_h="═" b_v="║"
        ;;
    2) # Modern Rounded
        b_tl="╭" b_tr="╮" b_bl="╰" b_br="╯" b_h="─" b_v="│"
        ;;
    3) # Dashed
        b_tl="+" b_tr="+" b_bl="+" b_br="+" b_h="-" b_v="|"
        ;;
    4) # Sharp Solid
        b_tl="┌" b_tr="┐" b_bl="└" b_br="┘" b_h="─" b_v="│"
        ;;
esac

local total_content_width total_box_width total_box_height box_start_col
local border_pad=5
if [ "$BORDER_STYLE" -gt 0 ]; then
    total_content_width=$(( inner_width + border_pad + border_pad ))
    total_box_width=$(( total_content_width + 2 ))
    total_box_height=$(( height + 2 ))
    box_start_col=$(( (TERM_COLS - total_box_width) / 2 ))
else
    total_content_width=$inner_width
    total_box_width=$inner_width
    total_box_height=$height
    box_start_col=$(( (TERM_COLS - inner_width) / 2 ))
fi

# Calculate vertical spacing
local date_rows=0
if [ "$SHOW_DATE" -eq 1 ]; then
    date_rows=3
elif [ "$SHOW_DATE" -eq 2 ]; then
    date_rows=2
fi

local total_full_height=$(( total_box_height + date_rows ))
local start_row=$(( (TERM_ROWS - total_full_height) / 2 ))
[ "$start_row" -lt 1 ] && start_row=1
[ "$box_start_col" -lt 1 ] && box_start_col=1

local current_cursor_row=$start_row

# 3. Draw Date (if enabled)
if [ "$SHOW_DATE" -eq 1 ]; then
    local date_col=$(( (TERM_COLS - ${#date_val}) / 2 ))
    [ "$date_col" -lt 1 ] && date_col=1
    printf "%s[%d;%dH%s[2K%s%s%s" "$ESC" "$current_cursor_row" "$date_col" "$ESC" "$COLOR_DATE_SUB" "$date_val" "$COLOR_RESET"
    current_cursor_row=$((current_cursor_row + 1))

    local day_name_upper
    day_name_upper=$(echo "$day_name" | tr "[:lower:]" "[:upper:]")
    local spaced_day=""
    for ((i=0; i<${#day_name_upper}; i++)); do
        spaced_day="${spaced_day}${day_name_upper:$i:1} "
    done
    spaced_day="${spaced_day% }"
    local day_col=$(( (TERM_COLS - ${#spaced_day}) / 2 ))
    [ "$day_col" -lt 1 ] && day_col=1
    printf "%s[%d;%dH%s[2K%s%s%s" "$ESC" "$current_cursor_row" "$day_col" "$ESC" "$COLOR_DATE" "$spaced_day" "$COLOR_RESET"
    current_cursor_row=$((current_cursor_row + 2))
elif [ "$SHOW_DATE" -eq 2 ]; then
    local compact_str="${day_name}, ${date_val}"
    local date_col=$(( (TERM_COLS - ${#compact_str}) / 2 ))
    [ "$date_col" -lt 1 ] && date_col=1
    printf "%s[%d;%dH%s[2K%s%s%s" "$ESC" "$current_cursor_row" "$date_col" "$ESC" "$COLOR_DATE" "$compact_str" "$COLOR_RESET"
    current_cursor_row=$((current_cursor_row + 2))
fi
```

```
    # 4. Draw Top Border (if border enabled)
    if [ "$BORDER_STYLE" -gt 0 ]; then
        local border_line=""
        for ((b=0; b<total_content_width; b++)); do
            border_line="${border_line}${b_h}"
        done
        local top_border="${b_tl}${border_line}${b_tr}"
        local bot_border="${b_bl}${border_line}${b_br}"

        printf "%s[%d;%dH%s[2K%s%s%s" "$ESC" "$current_cursor_row" "$box_start_col" "$ESC" "$COLOR_BORDER" "$top_border" "$COLOR_RESET"
        current_cursor_row=$((current_cursor_row + 1))
    fi

    # 5. Draw Clock Rows
    for ((l=1; l<=height; l++)); do
        local row_content="${lines[l]}"
        local side_text=""

        if [ "$FORMAT_12H" -eq 1 ] && [ "$l" -eq "$height" ]; then
            side_text="  ${COLOR_AMPM}${ampm_str}${COLOR_RESET}"
        fi

        if [ "$BORDER_STYLE" -gt 0 ]; then
            local border_side_pad="     "
            if [ "$FORMAT_12H" -eq 1 ] && [ "$l" -eq "$height" ]; then
                border_side_pad="  ${COLOR_AMPM}${ampm_str}${COLOR_RESET} "
            fi

            printf "%s[%d;%dH%s[2K%s%s%s     %s%s%s%s%s" \
                "$ESC" "$current_cursor_row" "$box_start_col" "$ESC" \
                "$COLOR_BORDER" "$b_v" "$COLOR_RESET" \
                "${COLOR_DIGIT}${row_content}${COLOR_RESET}" \
                "$border_side_pad" \
                "$COLOR_BORDER" "$b_v" "$COLOR_RESET"
        else
            printf "%s[%d;%dH%s[2K%s%s" \
                "$ESC" "$current_cursor_row" "$box_start_col" "$ESC" \
                "${COLOR_DIGIT}${row_content}${COLOR_RESET}" \
                "$side_text"
        fi
        current_cursor_row=$((current_cursor_row + 1))
    done

    # 6. Draw Bottom Border (if border enabled)
    if [ "$BORDER_STYLE" -gt 0 ]; then
        printf "%s[%d;%dH%s[2K%s%s%s" "$ESC" "$current_cursor_row" "$box_start_col" "$ESC" "$COLOR_BORDER" "$bot_border" "$COLOR_RESET"
        current_cursor_row=$((current_cursor_row + 1))
    fi

    # 7. Clean Muted Grey Controls Bar
    if [ "$SHOW_HELP" -eq 1 ]; then
        local help_msg="[T] 12/24H   [S] Sec   [C] Color: $THEME_NAME   [F] Font   [B] Border   [D] Date   [H] Close   [Q] Quit"
        local help_col=$(( (TERM_COLS - ${#help_msg}) / 2 ))
        [ "$help_col" -lt 1 ] && help_col=1
        printf "%s[%d;%dH%s[2K%s%s%s" "$ESC" "$((TERM_ROWS - 1))" "$help_col" "$ESC" "$COLOR_DIM" "$help_msg" "$COLOR_RESET"
    else
        local hint="[H] Controls   [Q] Quit"
        local hint_col=$(( (TERM_COLS - ${#hint}) / 2 ))
        [ "$hint_col" -lt 1 ] && hint_col=1
        printf "%s[%d;%dH%s[2K%s%s%s" "$ESC" "$((TERM_ROWS - 1))" "$hint_col" "$ESC" "$COLOR_DIM" "$hint" "$COLOR_RESET"
    fi
}

# ------------------------------------------------------------------------------
# Input Processing
# ------------------------------------------------------------------------------
process_key() {
    local key="$1"
    [ -z "$key" ] && return 1

    if [ "$SHOW_HELP" -eq 1 ]; then
        case "$key" in
            h|H|$'\e'|" "|$'\n')
                SHOW_HELP=0
                printf "%s[2J" "$ESC"
                return 0
                ;;
        esac
    fi

    case "$key" in
        q|Q) cleanup ;;
        $'\e') cleanup ;;
        t|T) FORMAT_12H=$(( 1 - FORMAT_12H )); printf "%s[2J" "$ESC" ;;
        s|S) SHOW_SECONDS=$(( 1 - SHOW_SECONDS )); printf "%s[2J" "$ESC" ;;
        d|D) SHOW_DATE=$(( (SHOW_DATE + 1) % 3 )); printf "%s[2J" "$ESC" ;;
        c|C) COLOR_THEME=$(( (COLOR_THEME + 1) % 6 )) ;; # Instant recolor without screen clear
        f|F) FONT_STYLE=$(( (FONT_STYLE + 1) % 3 )); printf "%s[2J" "$ESC" ;;
        b|B) BORDER_STYLE=$(( (BORDER_STYLE + 1) % 5 )); printf "%s[2J" "$ESC" ;;
        h|H|\?) SHOW_HELP=$(( 1 - SHOW_HELP )); printf "%s[2J" "$ESC" ;;
        *) return 1 ;;
    esac
    return 0
}

# ------------------------------------------------------------------------------
# Main Event Loop
```

```
# ------------------------------------------------------------------------------
update_term_size
printf "%s[2J" "$ESC"

blink=1
tick=0
render_clock "$blink"

while true; do
    raw_in=$(dd bs=10 count=1 2>/dev/null || true)
    if [ -n "$raw_in" ]; then
        if process_key "$raw_in"; then
            render_clock "$blink"
            tick=0
        fi
    fi

    sleep 0.05
    tick=$((tick + 1))

    # Every 10 ticks (0.5 seconds), update time and blink
    if [ "$tick" -ge 10 ]; then
        tick=0
        blink=$(( 1 - blink ))
        render_clock "$blink"
    fi
done
```